**UNIVERSIDADE KIMPA VITA**

**INSTITUTO SUPERIOR POLITÉCNICO DO UÍGE**

**CURSO DE LICENCIATURA EM ENGENHARIA INFORMÁTICA**

**LINGUAGEM DE PROGRAMAÇÃO VI**

# RELATÓRIO TÉCNICO: Análise Estatística De Dados De Saúde

**POR:**

**Ariel Alfredo da Cunha Manuel - Francisco Bráulio Figueiredo António - João José António da Silva - Kumbi Manuel Domingos - Matondo Kuanzambi Samuel João - Nzongo Oceano Nsumbo Manuel - Pedro Bengui.**

**Startup:** PROGRAMATHORs

**Tecnologias:** Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn,Ydata

**Docente da Disciplina:**

**Moyo Kanivengidio, MSc.**

**UÍGE, ABRIL DE 2026**

# 0. INTRODUÇÃO

As árvores são estruturas de dados hierárquicas amplamente utilizadas na Ciência da Computação para representar decisões, relações entre elementos e processos de busca. Dentro do contexto de Machine Learning, as árvores de decisão são modelos supervisionados que utilizam essa mesma estrutura para classificação de dados.

Neste trabalho, realiza-se uma análise da complexidade computacional de dois algoritmos de árvores de decisão: ID3, baseado em entropia, e CART, baseado no índice de Gini. O estudo utiliza um dataset de recomendação de produtos femininos, permitindo observar de forma gráfica e prática a formação da árvore, o número de nós, a profundidade e o custo computacional.

# 1. Objectivos

## 1.1 Objectivo Geral

Comparar a complexidade estrutural e temporal dos algoritmos ID3 e CART.

## 1.2 Objectivos Específicos

- Construir árvores de decisão a partir do dataset
- Medir número de nós
- Medir profundidade
- Comparar tempo de treinamento
- Relacionar resultados com a complexidade teórica

# 2. FUNDAMENTAÇÃO TEÓRICA

Uma árvore binária é uma estrutura hierárquica composta por nós, onde cada nó possui no máximo dois filhos: esquerdo e direito. Os principais conceitos usados neste trabalho são:

- Raiz: nó inicial
- Nós internos: nós de decisão
- Folhas: classes finais
- Altura: maior caminho entre raiz e folha
- Profundidade: nível máximo alcançado

## 2.1. **Algoritmo ID3 (**Iterative Dichotomiser 3)

O ID3 utiliza entropia para escolher o melhor atributo de divisão. O ID3 é um algoritmo de classificação e foi desenvolvido em 1986 por Ross Quinlan aceitando apenas dados categóricos. O algoritmo possui uma estratégia gananciosa na hora de escolher o melhor atributo em cada iteração, selecionando aquele que tem maior ganho de informação ou menor entropia, ou seja, maior pureza.

- Vantagem: maior interpretabilidade
- Desvantagem: pode gerar árvores mais profundas

## 2.2 Algoritmo CART

O CART utiliza o índice de Gini.

- Vantagem: árvores geralmente mais balanceadas
- Desvantagem: ligeiramente menos interpretável em alguns cenários

## 3. METODOLOGIA

Foi utilizado o dataset `mhia_dataset_treino.csv`, contendo atributos relevantes para recomendação de produtos.

- Atributos utilizados

- idade
- budget
- estilo
- ocasião
- frequência de compra

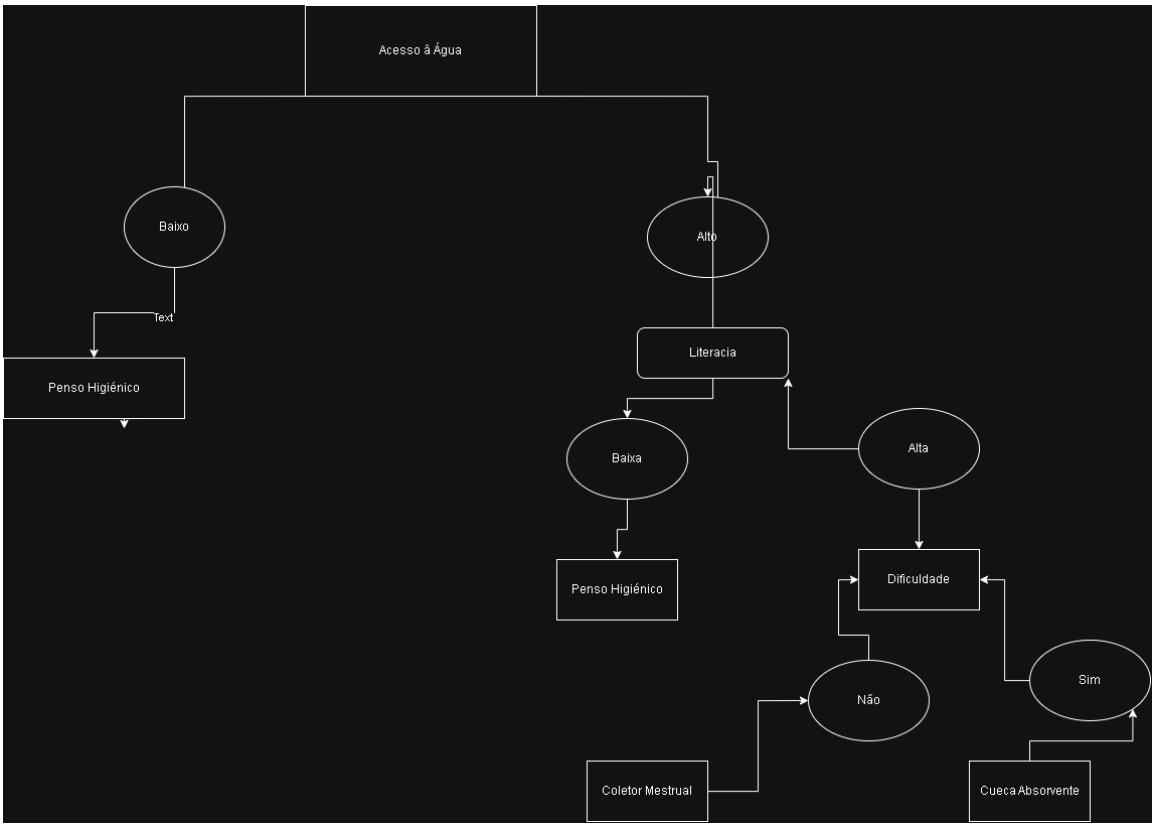
- Classe alvo

- Produto

Os algoritmos foram implementados em Python com a biblioteca **`scikit-learn`**

## 4. IMPLEMENTAÇÃO

4.1 Diagrama




**Interpretação:**

**A Árvore mostra que o atributo acesso à água é o principal fator de decisão. A partir dele, atributos como literacia e dificuldade de manuseio refinam a recomendação, conduzindo a diferentes produtos conforme o contexto da utilizadora. Cada caminho representa uma regra de decisão clara e interpretável.**

## 4.2 **Entropia E Ganho De Informação**

### 4.2.1 Entropia

A entropia de um conjunto pode ser definida como sendo o grau de pureza desse conjunto. Este conceito emprestado pela Teoria da Informação define a medida de "falta de informação", mais precisamente o número de bits necessários, em média, para representar a informação em falta, usando codificação óptima.

Vamos assumir um exemplo didático com **30 mulheres**:

- 12 usam **penso**
- 10 usam **cueca absorvente**
- 8 usam **coletor**

A fórmula é:

H(S)=−∑pi*$log_2$( (pi)

Substituindo:

H(S)=−(12/30)$log_2$(12/30)−(10/30) $log_2$(10/30)−(8/30) $log_2$ (8/30)
H(S)=-(12/30) $log_2$ (12/30)-(10/30) $log_2$ (10/30)-(8/30) $log_2$ (8/30)
H(S)=−(12/30) $log_2$(12/30)−(10/30) $log_2$ (10/30)−(8/30) $log_2$ (8/30)

H(S)=−(12/30) $log_2$ (12/30)−(10/30) $log_2$ (10/30)−(8/30) $log_2$ (8/30)

Resultado aproximado:

H(S)≈1.57

Interpretação:

O dataset está bastante misturado entre os 3 produtos.

### 4.2.2 Ganho Dos Atributos

Temos 30 utilizadoras.

- Água baixa (10 casos)

Distribuição:

- 9 → **Penso higiénico**
- 1 → **Cueca absorvente**

- 0 → **Coletor**

A fórmula é:

H(S)=−∑pi∗ $log_2$(pi)

Substituindo:

H(água baixa)=$-\left(\frac{9}{10}\right) log_2 \left(\frac{9}{10}\right)-\left(\frac{1}{10}\right) log_2 \left(\frac{1}{10}\right)$

Agora resolvendo:

$log_2 \left(\frac{9}{10}\right)\approx-0.152$ -0.152
$log_2 0.1)\approx-3.322$

Então:

H=−(0.9×−0.152)−(0.1×−3.322)
H=0.1368+0.3322
H=0.1368+0.3322
H≈0.469

**Entropia da água baixa = 0.47**

Grupo 2 → água alta (20)

- 3 penso
- 9 cueca
- 8 coletor

H(água alta)=$-\left(\frac{3}{20}\right) log_2 \left(\frac{3}{20}\right)-\left(\frac{9}{20}\right) log_2 \left(\frac{9}{20}\right)-\left(\frac{8}{20}\right) log_2 \left(\frac{8}{20}\right)$

Agora resolvendo:

$log_2 \left(\frac{3}{20}\right)\approx-2.737$
$log_2 \left(\frac{9}{20}\right)\approx-1.152$
$log_2 \left(\frac{8}{20}\right)\approx-1.322$

Então:

H=−(0.15×−2.737)−(0.45×−1.152)−(0.40×−1.322)

H=−(0.15×−2.737)−(0.45×−1.152)−(0.40×−1.322)

H=0.4106+0.5184+0.5288

H≈1.458

Entropia ≈ **1.53**

**Ganho**

$$\text{Gain}=H(S)-\left(\left(\frac{10}{30}\right)\times 0.469+\left(\frac{20}{30}\right)\times 1.458\right)$$

$$\text{Gain}=1.57-\left(\left(\frac{10}{30}\right)\times 0.469+\left(\frac{20}{30}\right)\times 1.458\right)$$

Gain=1.57−(0.156+0.972)
Gain=1.57−1.128
Gain=1.57−1.128
Gain≈0.442

**Ganho do atributo acesso à água = 0.44**

Interpretação**:**
Acesso à água tem ganho alto e é excelente candidato para **raiz da árvore**.

4.2- Código Python

```
import pandas as pd
import time
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt

# Carregar dataset
df = pd.read_csv("mhia_dataset_treino.csv")
print(df)
# Codificar dados categóricos
for col in df.select_dtypes(include="object").columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])

X = df.drop("produto", axis=1)
y = df["produto"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# ID3
```

```
id3 = DecisionTreeClassifier(criterion="entropy", max_depth=3)
start = time.time()
id3.fit(X_train, y_train)
tempo_id3 = time.time() - start
y_pred=id3.predict(X_test)
y_pred_id3prob=id3.predict_proba(X_test)

# CART
cart = DecisionTreeClassifier(criterion="gini", max_depth=3)
start = time.time()
cart.fit(X_train, y_train)
tempo_cart = time.time() - start
y_pred=cart.predict(X_test)
y_pred_prob=cart.predict_proba(X_test)
print("ID3 nós:", id3.tree_.node_count)
print("CART nós:", cart.tree_.node_count)
print(f"Tempo ID3: {tempo_id3}")
print(f"Tempo Cart: {tempo_cart}")

plt.figure(figsize=(14,8))
plot_tree(id3, feature_names=X.columns, filled=True)
plt.title("Árvore ID3")
plt.show()
```

**Resultados**:

Após a execução, devem ser analisadas as seguintes métricas:

| Algoritmo | Nº Nós | Tempo |
|---|---|---|
| ID3 | 9 | 0.09700155258178711 |
| CART | 11 | 0.02699732780456543 |

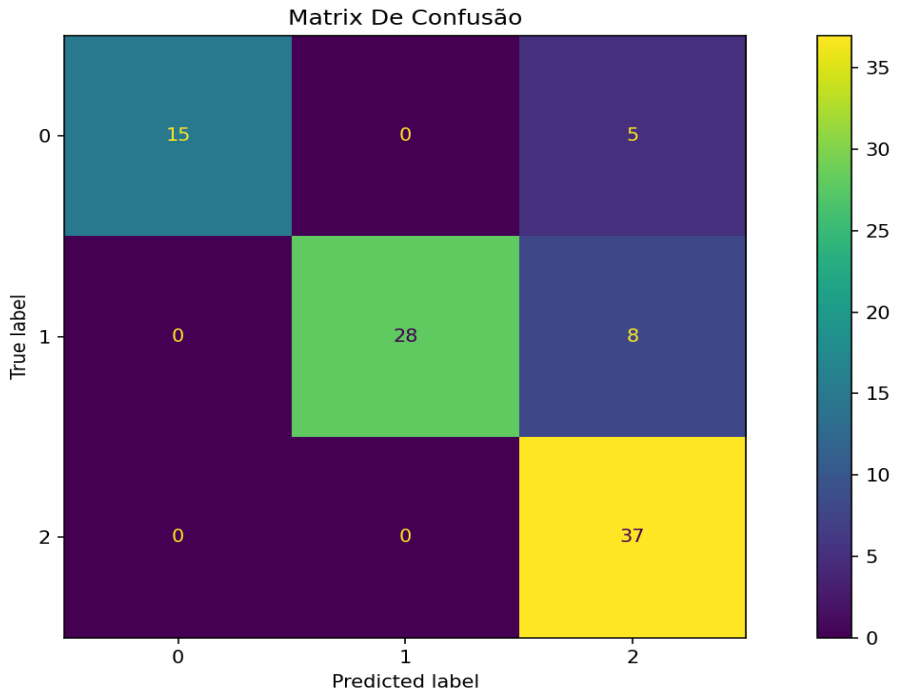
# 5.Métricas de Avaliação

## 5.1 Matrix De Confusão

| | Valor Previsto | | |
|---|---|---|---|
| Valor Real | | Y=1 | Y=0 |
| | Y=1 | Verdadeiro Positivo (VP) | Falso Negativo (FN) |
| | Y=0 | Falso Positivo (FP) | Verdadeiro Negativo (VN) |

Interpretação:

- Verdadeiro Positivo (VP): classe de interesse classificada corretamente;
- Falso Positivo (FP): classe de não interesse classificada como classe de interesse; • Verdadeiro Negativo (VN): classe de não interesse classificada corretamente;
- Falso Negativo (FN): classe de interesse classificada como classe de não interesse.




## 6. Complexidade Computacional

5.1 Construção Da Árvore

$O(n * f * log_2 n)$

Onde:

- n=Número de Amostras
- f=Número de Atributos

## 7. CONCLUSÃO

A análise mostrou que árvores de decisão são um excelente caso de estudo para complexidade de algoritmos, pois permitem visualizar nós, folhas, profundidade e altura.

A comparação entre ID3 e CART demonstrou como diferentes critérios de divisão impactam diretamente a estrutura da árvore e o custo computacional.